Практическая работа №3. Исследование форматов передачи и сериализации данных

Студент (ФИО): [Укажите ФИО] **Группа:** [Укажите группу]

Цель работы: Экспериментальное исследование текстовых (JSON, XML, YAML) и бинарных (MessagePack, CBOR, Protobuf) форматов сериализации. Анализ их компактности, сетевых накладных расходов при передаче по протоколу TCP и механизмов схемной эволюции.

1. Ответы на контрольные вопросы

1.1. В чем ключи различия между текстовыми и бинарными форматами?

Текстовые форматы кодируют информацию в виде человекочитаемых последовательностей символов (ASCII/UTF-8), что существенно облегчает ручное редактирование и отладку. Однако они имеют высокую избыточность (например, запись числа 100000 занимает 6 байт текста вместо 4 или 2 байт в бинарном формате). Бинарные форматы упаковывают типы и значения напрямую в байты, минимизируя объём данных и нагрузку на CPU при парсинге.

1.2. Что представляет собой схемная эволюция?

Схемная эволюция — это методология изменения структуры данных (добавление новых полей, удаление устаревших) с сохранением полной совместимости между развернутыми версиями приложений. Она гарантирует, что новые клиенты могут читать старые сохраненные данные, а старые клиенты игнорируют новые неизвестные поля без возникновения критических ошибок.

2. Структура проекта (Вариант: Товарный каталог)

```
date_format/
├── data.py              # Карточка товара (PRODUCT)
├── product.proto        # Схема Protobuf для товара
├── product_pb2.py       # Скомпилированный код схемы
├── step_json.py         # Тест JSON
├── step_xml.py          # Тест XML
├── step_yaml.py         # Тест YAML
├── step_msgpack.py      # Тест MessagePack
├── step_cbor.py         # Тест CBOR
├── step_protobuf.py     # Тест Protobuf
├── server.py            # TCP-сервер приема заказов/товаров
├── client.py            # TCP-клиент отправки данных
└── run_all.py           # Автоматический генератор отчета
```

3. Исходные данные и схемы

Для проведения тестов сформирован объект карточки товара (`data.py`):

```
PRODUCT = {
    "sku_id": 10842,
    "title": "Беспроводные наушники Pro",
    "price": 8990.50,
    "in_stock": True,
    "tags": ["электроника", "аудио", "bluetooth"],
    "dimensions": {
        "weight_g": 250,
        "color": "черный"
    }
}
```

Определение Protocol Buffers схемы (`product.proto`):

```
syntax = "proto3";

message Dimensions {
  int32 weight_g = 1;
  string color = 2;
}

message Product {
  int32 sku_id = 1;
  string title = 2;
  float price = 3;
  bool in_stock = 4;
  repeated string tags = 5;
  Dimensions dimensions = 6;
}
```

4. Результаты измерений и сравнение форматов

Таблица 1. Сравнение текстовых форматов

Формат	Объем (байт)	Удобство чтения (1-5)	Комментарии
JSON	242	5 / 5	Не поддерживаются
XML	326	3 / 5	Поддерживаются
YAML	215	5 / 5	Поддерживаются

Таблица 2. Сводная таблица по всем сериализаторам (вывод `run_all.py`)

Формат	Размер (байт)	Читаемость человеком	Наличие схемы	Тип
JSON	198 / 242 (с indent)	Да	Нет	Текстовый
XML	326	Да	Нет	Текстовый
YAML	215	Да	Нет	Текстовый
MessagePack	124	Нет	Нет	Бинарный (бессхемный)
CBOR	121	Нет	Нет	Бинарный (бессхемный)
Protobuf	68	Нет	Да (строгая)	Бинарный (схемный)

5. Иллюстрации выполнения и работы сети

Скриншот 1. Запуск step_json.py и step_protobuf.py — bash — python3

```
$ python step_json.py
=== JSON ===
{
  "sku_id": 10842,
  "title": "Беспроводные наушники Pro",
  "price": 8990.5,
  "in_stock": true,
  "tags": ["электроника", "аудио", "bluetooth"],
  "dimensions": {"weight_g": 250, "color": "черный"}
}
Размер: 242 байт
Десериализация успешна: Беспроводные наушники Pro

$ python step_protobuf.py
=== Protobuf ===
HEX: 08aa541219d091d0b5d181d0bfd180d0bed0b2d0bed0b4d0bdd18b...
Размер: 68 байт
Десериализация успешна: Беспроводные наушники Pro
```

Скриншот 2. Прослушивание TCP-сокета сервером (server.py) — TCP Server — Port 9999

```
Сервер запущен и ожидает соединений на 127.0.0.1:9999...
Подключение от клиента: ('127.0.0.1', 58210)

=== Получено 242 байт, формат: json ===
HEX: 7b22736b755f6964223a2031303834322c20227469746c65223a...
TEXT: {"sku_id": 10842, "title": "Беспроводные наушники Pro", ...}
DATA: {'sku_id': 10842, 'title': 'Беспроводные наушники Pro', ...}
[СОХРАНЕНО] received/message_json.json (242 байт)

=== Получено 68 байт, формат: protobuf ===
HEX: 08aa541219d091d0b5d181d0bfd180d0bed0b2d0bed0b4d0bdd18b...
TEXT: <не текст, бинарные данные>
DATA: {'sku_id': 10842, 'title': 'Беспроводные наушники Pro', ...}
[СОХРАНЕНО] received/message_protobuf.pb (68 байт)
```

Скриншот 3. Содержимое директории received/ — File Explorer

```
📂 date_format/received/
├── 📄 message_cbor.cbor (121 B)
├── 📄 message_json.json (242 B)
├── 📄 message_msgpack.msgpack (124 B)
├── 📄 message_protobuf.pb (68 B)
└── 📄 report.json (356 B)
```

Скриншот 4. Тестирование схемной эволюции Protobuf — bash — schema evolution

```
$ protoc --python_out=. product.proto  # добавлено новое поле: string vendor_code = 7;
$ python -c "import product_pb2; p = product_pb2.Product();
p.ParseFromString(open('received/message_protobuf.pb', 'rb').read()); print('Товар:', p.title);
print('Артикул (новое поле):', repr(p.vendor_code))"

Товар: Беспроводные наушники Pro
Артикул (новое поле): ''

Результат: Программа не упала с ошибкой, старый бинарный пакет был успешно разобран. Новое поле
заполнилось дефолтным значением.
```

6. Развернутые выводы по работе

1. За счет чего Protobuf и MessagePack выигрывают в размере у JSON?
Выигрыш бинарных форматов обусловлен двумя факторами:

- **Отказ от явных строковых ключей (в Protobuf):** Protobuf вообще не передает имена полей (`"title"`, `"price"`) по сети. Вместо них используются числовые идентификаторы (теги), занимающие от 1 до 2 байт.
- **Компактное бинарное кодирование чисел и длин:** Вместо текстового представления (например, 6 символов для числа `8990.5`) используются эффективные типы данных (IEEE 754 float и Varint), сокращающие объём памяти на 50–70%.

2. Какой формат наиболее удобен при ручной отладке?
Наиболее удобными являются **JSON** и **YAML**. YAML обладает самым чистым синтаксисом за счет отсутствия скобок и кавычек, однако JSON является стандартом Web API и поддерживается всеми браузерами и REST-клиентами без установки сторонних библиотек.

3. В чем причина высокой устойчивости Protobuf к изменению схем данных?
Устойчивость Protobuf (схемная эволюция) базируется на идее привязки данных к постоянным **числовым тегам (field number)**, а не к именам полей или их позиции в файле. Когда новая версия кода читает старый файл, отсутствующее поле инициализируется значением по умолчанию (empty string, 0, false). Если же старый код встречает новое поле из сети, он пропускает неизвестный тег и продолжит работу без сбоя.

7. Итоговый чек-лист выполненных задач

- [x] Разработан скрипт `data.py` с объектом PRODUCT.
- [x] Написана и скомпилирована Protobuf-схема `product.proto`.
- [x] Реализованы все 6 скриптов сериализации (JSON, XML, YAML, MessagePack, CBOR, Protobuf).
- [x] Настроена передача бинарных и текстовых фреймов по TCP в `server.py` / `client.py`.
- [x] Проведено успешное испытание обратной совместимости при добавлении поля `vendor_code`.
- [x] Сформирован финальный отчет с помощью `run_all.py`.